

Shorts 1: Dictionaries

Dictionaries are useful for key-value pairs

-key: Some name used to access a particular value inside the dictionary

ex. `def main():`

```
... spacecraft = {"name": "Voyager 1", "distance": 163}
... print(create_report(spacecraft))
```

a dictionary (dict) with
values "name" and "distance"

`def create_report(spacecraft):`

```
... return f"Name: {spacecraft["name"]}"
```

```
... distance: {spacecraft["distance"]}
```

key names indicated
utilizes the "spacecraft" dict

`main()`

There is a method called "get" that tries to access some key. If the key does not exist, we will get another value that is specified.

ex. `def main():`

```
... tool = {"name": "telescope"}
... print(create_report(tool))
```

name: telescope
distance: unknown

`def create_report(tool):`

```
... return f"Name: {tool["name"]}"
```

```
... distance: {tool.get("distance", "unknown")}
```

2 Common ways to add keys:

1. `terms = {"word1": "def1", "word2": [int]}`

2. `terms["word3"] = "def2"`

There is a third method to add keys:

- Update() : tool.update("usage": "docking", "type": "poop")
- takes as an argument/input, another dictionary, but really takes dictionary's keys and values and adds them to the previous dictionary

Other utility feature of dict:

ex 1. distances = {
 ... "Vaj 1": 163,
 ... "Vaj 2": 136,
 ... "Pion 10": 80,
 ... "New H": 58,
 ... "Pion 11": 44
}

This dictionary represents multiple spacecrafts while also defining the relationship between spacecrafts

def main():
 ... for name in distances.keys():
 ... print(f"{name} is {distances[name]} in AU from Earth")
main()

all keys become a return value for this

We will try to convert the units from AU to meters

ex 2. def main():
 ... for distance in distances.values():
 ... print(f"{distance} AU is {convert(distance)} m")
 def convert(au):
 ... return au * 149597870700
main()

formatted print function
takes the values of each key from "distances" function and also runs the "convert" function with distances values as inputs.

ex1 output:

Voy 1 is 163 in AU from Earth
Voy 2 is 136 in AU from Earth
Pion 10 is 80 in AU from Earth
New H is 58 in AU from Earth
Pion 11 is 44 in AU from Earth

ex2 output:

163 AU is 24 ~~mm~~ 100 m
136 AU is 20 ~~mm~~ 200 m
80 AU is 11 ~~mm~~ 000 m
58 AU is 86 ~~mm~~ 600 m
44 AU is 65 ~~mm~~ 800 m

Shorts 2: Dictionary Methods

functions we can use to manipulate dictionaries

ex. Spelling bee

Words = {"PAIR": 4, "HAIR": 4, "CHAIR": 5}

dict with list of words possible

def main():

... while len(Words) > 0:

... print(f"{len(Words)} words left")

... guess = input("guess a word:")

"len" tells how many keys

there are, and f-string

formats it into the print function

... if guess in Words.keys():

... points = Words.pop(guess)

... print(f"scored {points} points")

"pop()" returns the value

associated with the key, and

remove the key from the dictionary.

"keys()" identifies the key in the dictionary.

In regards to the "if" statement, if the input value "guess" is in the dictionary "Words" and matches a certain key within the dictionary, then the following block will run.

example terminal:

3 words left

guess a word: PAIR

Scored 4 points

2 words left

guess a word: HAIR

Scored 4 points

1 words left

guess a word: CHAIR

Scored 5 points

ex. If the user guesses a "Super word" that clears the game when entered:
if guess == "GRAPHIC":

Words.clear()

print("win")

".clear()" removes not just
"GRAPHIC", but all the keys in
the dictionary

To show the user all possible words and their number of points, i.e.,
iterating over each key and value in the dictionary:

".items()" gives back a set where it gives both the key and value
associated with the dictionary

for [key], [value] in dict.items():

ex. for word, points in Words.items():

print(f"{word} was worth {points} points.")

PAIR was worth 4 points.

HAIR was worth 4 points.

CHAIR was worth 5 points.

GRAPHIC was worth 7 points.

Shorts 3: for loops

for loops are good for when we know how many times we want to iterate something.

```
ex. names = ["poo", "poob", "poooy", "peebe"]  
for i in range(len(names)):  
    print(names[i])
```

Poo

Poob

Poooy

Peebe

To make this more readable, we can iterate over any kind of list directly.

```
ex. names = ["poo", "poob", "poooy", "peebe"]  
for name in names:  
    print(name)
```

[Same result as above]

Shorts 4: Lists

In most cases, we want to add stuff to an existing list.

• `append()` adds a new element to the end of an existing list.

```
ex. names = ["poo", "poob"] | ["poo", "poob", "peebe"]  
names.append("peebe")  
print(names)
```

To add more than one value at a time, we don't use `append`.

```
ex. names.append(["peebe", "poooy"]) | ["poo", "poob", ["peebe", "poooy"]]
```

"`.append([list])`" will add the list itself to the targetted list.

"`.remove()`" can remove existing entries/values from the targetted list.

"`.extend()`" can extend a targetted list with values, but will really take each element of an inputted list and append it to the focused list.

ex. <code>names.remove(["pob", "poobag"])</code>	<code>["poo", "poob", "pob", "pegirl"]</code>
<code>names.extend(["pob", "pegirl"])</code>	

"`.insert()`" takes as its first argument the index at which we want to insert some given element.

ex. <code>names.remove("Pob")</code>	<code>["Pob", "poo", "poob", "pegirl"]</code>
<code>names.insert(0, "Pob")</code>	

we can flip the order using "`.reverse()`".

ex. <code>names.reverse()</code>	<code>["pegirl", "poob", "poo", "pob"]</code>
----------------------------------	---

Shorts 5: List and Dictionary Comprehensions

List comprehension is a way to build up a new list from some list that already exists.

apply list syntax: `listname = []`

apply a for loop: `listname = [ [Var] for [Var] in [otherlist] ]`

$listname = [var \text{ for } var \text{ in } otherlist]$

This gives me back the same list. For every element (var) in this list (otherlist), give me back that element (var) unchanged as a new list (listname). We can alter the new list (listname) by applying functions within the brackets:

ex: $listname = [var.lower() \text{ for } var \text{ in } otherlist]$

For every element (var) in this list (otherlist), give me back that element (var) changed so that it now gives me the element in lowercase as a new list (listname).

List Comprehensions is also used to filter lists.

ex. $listname = [var.lower() \text{ for } var \text{ in } otherlist \text{ if } len(var) > 4]$

For every element (var) in this list (otherlist), give me back that element (var) changed so that it now gives me the element in lowercase, IF that element has a length greater than 4, and put it in a new list (listname).

Dictionary Comprehension is generally the same thing: Iterate over every element in a list and build up a dictionary based on that list.

ex. $dictname = \{var: otherlist.count(var) \text{ for } var \text{ in } otherlist\}$

For every element (var) in this list (otherlist), give me a key called "var" and set the value of "var" to the count that the word appears.

Shorts 6: List Methods

List method is some function used to manipulate the data inside the lists.

ex. `history = []`

While True:

... `action = input("action: ")`

... if `action == "m2a":`

... `history.pop()` ← `.pop()` takes out and removes the last element of a list IF "m2a" is inputted

... else:

... `history.append(action)` ← `.append()` adds the inputted value into the list "history"

`print(history)`

ex. ... elif `action == "Restart":`

... `history.clear()` ← `.clear()` removes all elements of the specified list IF "Restart" is inputted

Shorts 7: String Slicing

Taking certain sections of a string using specified indices

ex. `Stringex = "123abc"`

`print(Stringex[0:3])`

`print(Stringex[:3])`

`print(Stringex[3:6])`

`print(Stringex[3:])`

`print(Stringex[-4:])`

123

123

abc

abc

3abc

Start at 0 index, end at 3rd index (excl.)

Start from start and go to 3rd index (excl.)

Start at 3rd index and go to 6th index (excl.)

Start at 3rd index and go to the end

Start at -4th index and go to the end

Str: 123abc

index: 0 1 2 3 4 5

→

also:

1 2 3 a b c

0 -5 -4 -3 -2 -1

← ←

Shorts 8: Tuples

Tuples are combinations of values that are assigned to another variable. They are ordered immutable sequence of elements.

ex. `yourlocation = (42.376, -71.115)`

Tuples list out values and package them together into a single variable

To get each individual value of a tuple, note the index of the value within the specified tuple.

ex. `print(f"latitude: {yourlocation[0]}")`
`print(f"longitude: {yourlocation[1]}")`

latitude: 42.376
longitude: -71.115

Tuples have a feature which enables them to be "unpacked". "Unpack" means to take individual components and make them their own distinct variables.

ex. `latitude, longitude = yourlocation`
`print(f"latitude: {latitude}")`
`print(f"longitude: {longitude}")`

latitude: 42.376
longitude: -71.115

The difference between a tuple and a list is that a tuple cannot be altered in any way, while a list can. The benefit of a tuple comes from the intention that the values within a tuple will not change; there will be a much more efficient way to represent a collection of data.

To see how much space each data structure takes up in memory: import a module called "sys"

```

ex. import sys
def main():
    ... CoordTuple = (123, abc)
    ... CoordList = [123, abc]
    ... print(f"{sys.getsizeof(CoordTuple)} bytes")
    ... print(f"{sys.getsizeof(CoordList)} bytes")
main()

```

56 bytes

72 bytes

Both CoordTuple and CoordList have the exact same data, but Tuples are a more efficient way of representing it in terms of memory.

Shorts 9: While Loops

While loops are usually used when the amount of times needed to loop is unknown, but a loop is desired when a certain condition is True.

Good for randomness

A while loop will end when the condition becomes False, aka, the condition is no longer met.